DBMS LABORATORY MINI-PROJECT | ACADEMIC YEAR 2025–2026

StreamSphere

OTT Platform Management System

A Full-Stack Relational Database System with MySQL 8.0, Node.js REST API, and Modern React Dashboard

Institution & Department

DEPARTMENT:
[Department of Computer Science & Engineering]

COLLEGE / INSTITUTION:
[Your College / Institute Name Here]

Project Team Members

TEAM MEMBERS & ROLL NUMBERS:
1. [Student Name 1 — Roll No / Reg No]
2. [Student Name 2 — Roll No / Reg No]
3. [Student Name 3 — Roll No / Reg No]

Degree: B.Tech Computer Science & Engineering (Year II)

Faculty & Evaluation

COURSE INSTRUCTOR / GUIDE:
[Prof. / Dr. Faculty Name Here]

LABORATORY COURSE:
Database Management Systems Lab
Course Code: [CS-302P / CS-XXXX]

1. INTRODUCTION & DOMAIN BACKGROUND

Understanding OTT Operations & Database Imperatives

Why media streaming platforms require strict relational consistency over simple flat stores

The OTT Streaming Paradigm

- Over-The-Top (OTT) media services stream films, episodic web series, and live television directly to consumers over the public Internet.
- Bypasses traditional broadcast, cable, and satellite television distributors.
- Examples: Netflix, Amazon Prime Video, Disney+ Hotstar, HBO Max.
- Exponential scale: modern platforms manage tens of thousands of catalog titles and hundreds of thousands of concurrent viewing streams.

Why Relational Integrity Matters

- Financial Billing Accuracy: Subscriptions involve recurring transactions; missing or duplicate records result in revenue loss or user disputes.
- Temporal Access Windows: Access must be strictly governed by start and expiration dates (end_date >= start_date).
- Playback Telemetry: Tracking stream progress (minutes watched) requires referential integrity across Users and Content.
- ACID Guarantees: Activation of paid plans requires atomic execution — updating prior subscriptions and creating new ones in a single transaction.

The StreamSphere Solution

- Relational-First Design: 7 interconnected tables modeled in MySQL 8.0 InnoDB engine.
- Zero Mock Data: Every dashboard card, genre bar, and report is computed live via SQL queries.
- Integrated SQL Objects: Utilizes MySQL Stored Procedures, Triggers, Views, and Check Constraints.
- Academic & Professional Rigor: Fully normalized schema (3NF/BCNF) paired with an enterprise REST API and modern UI.

Operational Challenges & Technical Project Objectives

Bridging the gap between raw media catalogs and reliable, normalized database management

Operational Challenges & Problem Statement

Data Redundancy in Catalogs: Unnormalized flat schemas store repetitive genre names and plan pricing strings, leading to high storage overhead and update anomalies.

Subscription Overlaps & Inconsistencies: Without atomic database procedures, users can hold conflicting active subscriptions or invalid expiration dates (e.g. end date earlier than start date).

Orphaned Records & Integrity Loss: Deleting a user or content title without defined foreign key deletion rules leaves dangling watch telemetry and unlinked ratings.

Academic Simulation Fallacy: Many DBMS academic projects rely on hardcoded JavaScript arrays or client-side mock storage, failing to demonstrate real SQL query execution.

Key Project Objectives & System Scope

Formally Normalized Schema: Design a 7-table relational schema meeting Third Normal Form (3NF) and BCNF standards to eliminate all update, insertion, and deletion anomalies.

Transactional Procedure Execution: Implement a MySQL Stored Procedure (`sp_register_subscription`) to atomically validate plans, calculate terms, and update active statuses.

Active Trigger-Based Enforcement: Implement server-side MySQL Triggers (`trg_validate_subscription_insert`) to block invalid dates and negative prices at the engine level.

Analytical Querying & Views: Create pre-aggregated SQL Views (`vw_content_performance`) and report queries demonstrating JOIN, GROUP BY, SUM, AVG, and HAVING.

Project Scope Boundary: Focuses on administrative operations, subscriber lifecycle, and telemetry analytics (excluding actual video file streaming and CDN bandwidth management).

3. SYSTEM ARCHITECTURE

Three-Tier Client-Server Architecture

End-to-end data flow: User Action → React Client → Express REST API → MySQL 8.0 Engine

CLIENT TIER (React + Vite) ──[HTTP / REST JSON]──> **APPLICATION TIER (Node + Express)** ──[Parameterized SQL]──> **DATABASE TIER (MySQL 8.0)**

Single Page Application (SPA) | Role-Based JWT Auth | Prepared Statements (Anti-SQLi) | InnoDB ACID Engine

Client Tier: React 18 & Vite

- Vite Build Tool: Lightning-fast HMR and optimized production bundles.
- Component Architecture: Reusable views for Dashboard, Catalog, Subscriptions, and Reports.
- Lucide Icons & Modern CSS: Custom dark glassmorphism theme inspired by premier streaming platforms.
- Reactive State: Synchronous UI feedback on CRUD operations with toast notifications.

App Tier: Node.js & Express 5

- Express 5 REST API: Structured modular routes for users, content, subscriptions, and telemetry.
- Security & Authentication: One-way bcrypt password hashing with salt rounds and signed JWT authorization.
- Parameterized Prepared SQL: Zero string concatenation in queries, strictly preventing SQL Injection attacks.
- Environment Secrets: Isolated database credentials managed via .env configuration.

Data Tier: MySQL 8.0 InnoDB

- InnoDB Engine: Full ACID transactional safety, foreign key constraint enforcement, and row-level locking.
- 7 Relational Tables: High-performance indexed storage for catalogs, users, billing, and logs.
- Database Programmability: Pre-compiled Stored Procedures, Triggers, Views, and Stored Functions.
- Connection Pooling: Managed via mysql2/promise with connection limits and health checks.

Conceptual Entity-Relationship (ER) Model

Visual mapping of 7 core entities, primary keys, foreign keys, and relationship cardinalities

Implemented Entity Definitions & Keys

1. **Users:** **[user_id (PK)]** name, email (UK), password_hash, role, created_at
2. **Genres:** **[genre_id (PK)]** genre_name (UK), description
3. **Content:** **[content_id (PK)]** title, content_type, genre_id (FK), release_year, duration
4. **SubscriptionPlans:** **[plan_id (PK)]** plan_name (UK), price, duration_days, description
5. **Subscriptions:** **[subscription_id (PK)]** user_id (FK), plan_id (FK), start_date, end_date, status
6. **WatchHistory:** **[history_id (PK)]** user_id (FK), content_id (FK), watched_at, progress_minutes
7. **Ratings:** **[rating_id (PK)]** user_id (FK), content_id (FK), rating, review, UK(user, content)

Relationship Cardinality Matrix

- **Users (1) — (M) Subscriptions**
  One user purchases zero, one, or multiple subscription terms over time. Each subscription record belongs strictly to exactly one customer account.
- **SubscriptionPlans (1) — (M) Subscriptions**
  A commercial tier (e.g. Standard HD) is subscribed to by many users. Each active subscription is governed by one pricing tier.
- **Genres (1) — (M) Content**
  One genre category (e.g. Sci-Fi) classifies multiple movies and series. Each catalog item references a valid genre.
- **Users (M) — (N) Content [via WatchHistory]**
  Many-to-many relationship tracking viewing telemetry. Resolved by associative entity WatchHistory with progress minutes.
- **Users (M) — (N) Content [via Ratings]**
  Many-to-many relationship storing audience critiques. Governed by composite UNIQUE constraint (user_id, content_id).

Relational Schema & Referential Deletion Rules

Data types, integrity constraints, and academic justification for deletion behaviors

Relational Integrity & Check Constraints

1. PRIMARY KEY & UNIQUE CONSTRAINTS:
Surrogate integer PKs across all 7 tables. Unique business keys prevent duplicate email accounts, duplicate plan names, and multiple reviews per user-title pair.

2. CHECK CONSTRAINTS ENFORCED:
- Content: release_year BETWEEN 1900 AND 2100
- Plans: price >= 0.00 and duration_days > 0
- Subscriptions: amount_paid >= 0.00 and end_date >= start_date
- Ratings: rating BETWEEN 1.0 AND 5.0

Referential Integrity & Deletion Actions

1. ON DELETE RESTRICT (Data Protection Rules):
- Genres -> Content: Prevents deleting a genre if active movies or series reference it. Protects catalog structure.

- SubscriptionPlans -> Subscriptions: Prohibits deleting commercial pricing tiers that are referenced by active or historical subscription contracts.

2. ON DELETE CASCADE (Account Purge Rules):
- Users -> Subscriptions, WatchHistory, Ratings: When an account is removed, personal telemetry and billing transactions are purged to prevent orphaned references.
- Content -> WatchHistory, Ratings: Purging a movie removes its associated telemetry logs and reviews automatically.

Normalization Analysis & Anomaly Resolution

Formal proof of schema decomposition from Unnormalized Form (UNF) through 3NF and BCNF

First Normal Form (1NF)

Requirement: Atomic attribute values; eliminate repeating groups.

- Flat relation had repeating multi-valued fields for watched titles and purchased subscriptions.
- Decomposed into distinct entities (WatchHistory, Subscriptions). Every column has atomic scalar domain types.

Second Normal Form (2NF)

Requirement: In 1NF and no partial dependencies on candidate keys.

- All primary keys are single-attribute surrogate keys (user_id, content_id), making partial key dependency mathematically impossible.
- In Ratings with alternate key (user_id, content_id), rating and review depend fully on the complete key.

Third Normal Form (3NF)

Requirement: In 2NF and no transitive functional dependencies.

- Transitive dependency content_id -> genre_id -> genre_name eliminated by decomposing into Genres.
- subscription_id -> plan_id -> (price, duration) eliminated by isolating SubscriptionPlans. No non-prime attribute determines another non-prime attribute.

Boyce-Codd Normal Form (BCNF)

Requirement: For every functional dependency X -> Y, X is a superkey.

- Users: user_id and email are candidate keys.
- Genres: genre_id and genre_name are candidate keys.
- Plans: plan_id and plan_name are candidate keys.
- Ratings: rating_id and (user_id, content_id) are candidate keys. Schema achieves BCNF.

Complex SQL Queries, JOINs & Aggregate Reports

Actual parameterized queries executed live against MySQL 8.0 streamsphere_db

1. Content by Genre (JOIN & Conditional SUM)

SQL Query Statement:

```
SELECT g.genre_id, g.genre_name,
  COUNT(c.content_id) AS total_titles,
  SUM(CASE WHEN c.content_type = 'Movie'
      THEN 1 ELSE 0 END) AS movie_count,
  SUM(CASE WHEN c.content_type = 'Series'
      THEN 1 ELSE 0 END) AS series_count
    FROM Genres g
LEFT JOIN Content c ON g.genre_id = c.genre_id
  GROUP BY g.genre_id, g.genre_name
    ORDER BY total_titles DESC;
```

DBMS Viva Explanation:
Uses LEFT JOIN so genres with 0 titles are preserved. Combines COUNT with conditional SUM(CASE) to simultaneously calculate total assets, movies, and series in a single table pass.

2. Ratings & Reviews (GROUP BY & HAVING)

SQL Query Statement:

```
SELECT c.content_id, c.title, g.genre_name,
    COUNT(r.rating_id) AS total_reviews,
  ROUND(AVG(r.rating), 2) AS average_rating,
      MIN(r.rating) AS min_rating,
      MAX(r.rating) AS max_rating
        FROM Content c
  JOIN Genres g ON c.genre_id = g.genre_id
JOIN Ratings r ON c.content_id = r.content_id
GROUP BY c.content_id, c.title, g.genre_name
      HAVING COUNT(r.rating_id) >= 1
      ORDER BY average_rating DESC;
```

DBMS Viva Explanation:
Demonstrates 3-table INNER JOIN across Content, Genres, and Ratings. Uses aggregate functions AVG, MIN, MAX, COUNT, and filters aggregated groups using the HAVING clause.

8. ADVANCED DATABASE OBJECTS

Stored Procedure, Trigger, SQL View & Function

Server-side business logic and transactional abstractions implemented in MySQL 8.0

Stored Procedure: sp_register_subscription

- Parameters: IN p_user_id, IN p_plan_id, IN p_start_date, OUT p_sub_id, OUT p_message
- Transactional Atomicity: Uses START TRANSACTION, COMMIT, and EXIT HANDLER FOR SQLEXCEPTION with ROLLBACK.
- Business Rules: Validates user/plan existence, computes end_date = DATE_ADD(start_date, INTERVAL duration DAY), and auto-expires prior active subscriptions.

MySQL Trigger: trg_validate_subscription_insert

- Event: BEFORE INSERT ON Subscriptions FOR EACH ROW.
- Validation Logic: Checks if NEW.end_date < NEW.start_date or NEW.amount_paid < 0.00.
- Exception Signaling: Throws SIGNAL SQLSTATE '45000' with custom error message.
- Live Viva Test: Tested via the app's Reports page; attempting invalid dates immediately raises ER_SIGNAL_EXCEPTION.

SQL View: vw_content_performance

- Multi-Table Abstraction: Pre-aggregates Content, Genres, WatchHistory, and Ratings.
- Pre-Computed Metrics: Exposes total_views, total_ratings, and avg_rating.
- Performance & Security: Eliminates writing 4-table joins in dashboard queries and simplifies client access.

Stored Function: fn_calculate_user_watch_hours

- Deterministic Function: Takes p_user_id INT, queries SUM(progress_minutes) from WatchHistory.
- Mathematical Output: Returns ROUND(v_total_minutes / 60.0, 2) as DECIMAL(6,2).
- Modular Integration: Directly callable within SELECT projection queries across the database.

9. APPLICATION WALKTHROUGH

StreamSphere Application Modules & UI Showcase

Verified operational modules connected directly to MySQL backend REST endpoints

1. Operational Dashboard

Live KPI cards (Users, Catalog, Active Subscriptions, Revenue), interactive genre distribution bars, popular titles, and real-time playback telemetry feed.

[Screenshot: Live Screen in App]

2. Content Catalog Management

Media inventory table with live search by title, filters by genre and format (Movie/Series), release year sorting, and modal CRUD dialogs.

[Screenshot: Live Screen in App]

3. User & Access Control

Customer and admin profile management with role indicators, email uniqueness verification, and security guard preventing self-deletion.

[Screenshot: Live Screen in App]

4. Subscription Lifecycle

Interactive execution of Stored Procedure sp_register_subscription, automatic expiration handling, and remaining access days countdown.

[Screenshot: Live Screen in App]

5. Playback Telemetry & Reviews

Watch history logger tracking streaming duration in minutes, coupled with audience rating submission on a 1.0–5.0 star scale.

[Screenshot: Live Screen in App]

6. DBMS Lab & Viva Showcase

Dedicated demonstration screen with live SQL query inspector, SQL View explorer, Stored Procedure runner, and interactive Trigger tester.

[Screenshot: Live Screen in App]

10. TESTING & QUALITY ASSURANCE

Empirical Testing & Verification Results

100% verified test cases executed against MySQL database and REST API endpoints

VERIFIED TEST EXECUTION: ALL 15 AUTOMATED SYSTEM & DATABASE TESTS PASSED SUCCESSFULLY

Automated End-to-End Test Matrix

```
TC-01 | Database Connectivity | Verify MySQL connection pool and health check | Expected: HTTP 200, connected to streamsphere_db  [PASS]
TC-02 | Admin Authentication | Login with valid admin credentials       | Expected: HTTP 200, JWT token returned, role: Admin  [PASS]
TC-03 | Invalid Login Guard  | Submit incorrect password                | Expected: HTTP 401 Unauthorized, access rejected  [PASS]
TC-04 | Live Dashboard Stats | Query aggregate user, content, and revenue KPIs | Expected: HTTP 200, users=12, content=12, rev=₹10,938  [PASS]
TC-05 | Parameterized Filtering | Filter content by genre (Sci-Fi, genre_id=2) | Expected: HTTP 200, returns exactly 3 Sci-Fi titles  [PASS]
TC-06 | Stored Procedure     | Execute CALL sp_register_subscription(4, 1, ...) | Expected: HTTP 201, auto-expires prior sub, returns ID  [PASS]
TC-07 | MySQL Trigger Validation | Attempt invalid subscription (end_date < start_date) | Expected: Trigger intercepts, signals SQLSTATE 45000  [PASS]
TC-08 | SQL View Execution   | Query SELECT * FROM vw_content_performance | Expected: HTTP 200, 12 summarized rows returned  [PASS]
TC-09 | Referential Integrity | Delete genre referenced by active titles | Expected: HTTP 400 Bad Request (RESTRICT enforced)  [PASS]
TC-10 | Composite Unique Key | Submit review on existing user-content pair | Expected: HTTP 201, ON DUPLICATE KEY UPDATE executed  [PASS]
```

11. CONCLUSION & SCOPE

Project Summary, Academic Contributions & Future Scope

Reflecting on achievements, database engineering takeaways, and technical scalability

Key Project Achievements

- **Database-First Engineering:** Built a complete working system where every single view and report runs genuine SQL against MySQL 8.0.
- **Formal Normalization:** Demonstrated and verified schema decomposition up to 3NF/BCNF, eliminating update anomalies.
- **Active SQL Programmability:** Implemented and verified Stored Procedures, Triggers, Views, and Check Constraints.
- **Security & Integrity:** Enforced parameterized queries against SQL injection and utilized bcrypt password hashing.
- **Viva Demonstration Ready:** Full UI and backend running live with automated test confirmation.

Future Roadmap & Scalability Scope

- **CDN & HLS Video Streaming:** Integration of Amazon S3/CloudFront and HTTP Live Streaming (HLS) adaptive bitrate chunking.
- **ML Recommendation Engine:** Collaborative filtering matrix factorization and cosine similarity models based on watch history and ratings.
- **Commercial Payment Webhooks:** Integration with payment gateways (Stripe/Razorpay) for automated renewal events.
- **Multi-Profile Account Sharing:** Adding family profile partitions per user account with parental rating control PINs.

THANK YOU! | QUESTIONS & LIVE DEMONSTRATION

StreamSphere DBMS Mini-Project is ready for live SQL inspection, trigger tests, and viva evaluation.